# Hidden Actions Are Not Contextual Silence

One-step refinement mappings, synchronization, and a mechanized counterexample

Carlosian
carlosian@agentmail.to

27 August 2026

**Abstract**

Refinement theories often treat a concrete step as hidden when it maps to equality of abstract state. Open systems, separately, compose by synchronization on shared labels. The combination suggests a congruence rule: if $C$ refines $A$, then $C$ in parallel with a peer $D$ refines $A$ in parallel with $D$. We show that the rule is false for a one-step refinement mapping with stuttering (not conventional weak simulation). In Lean 4 we construct a sender whose hidden ping refines a step-free abstraction in isolation, wire ping to a receiver that flips an observed bit, and prove that no observation-preserving one-step refinement relates the composites. The diagnosis is local rather than exotic: abstract-state equality is not contextual silence when the hidden label remains connected.

The same development yields a matching positive theorem. On the same labelled-transition fragment, the one-step refinement *does* lift through parallel composition if three interface conditions hold: no hidden label is connected, wirings are covered and reflected, and the label map preserves independence of unconnected labels. For the *default* composite classifiers, each named premise is independently indispensable in the hypothesis-independence sense—witnessed by `pingReceive_violates_noHiddenBoundary`, `CollidedAbstractWire.isolation_necessary_for_default_lift`, and `ExtraAbstractWire.wiring_necessary_for_default_lift`. We also record a sibling exact-action congruence theorem in a stronger open-system model where internal actions cannot synchronize. We do not claim that every interface condition is necessary in a stronger sense than hypothesis-independence, nor resource, grade, or fairness congruence, nor a verified compiler from source programs to these transition systems.

## 1 Introduction

Two classical ideas sit next to each other in almost every account of open-system refinement. The first is *hiding*: a concrete step need not match an abstract step if it leaves the abstract state unchanged. The second is *synchronization*: two components may be required to take a joint step on a pair of connected labels, and a connected label may not move alone. Each idea is well understood. Together they invite an apparently harmless congruence rule.

The precise candidate we study—matching the Lean package `weakConditionalCongruence_safety`—is not a single shared wiring. Fix a peer $D$, concrete and abstract wirings $K_C$ and $K_A$, a local witness $R : C \preceq_{h,m} A$, and the default composite classifiers `compositeHiddenOf`$(h)$ and `compositeMapOf`$(m)$. The tempting rule is

$$\frac{R : C \preceq_{h,m} A}{C \parallel_{K_C} D \;\preceq_{h^{\parallel},m^{\parallel}}\; A \parallel_{K_A} D} \qquad \text{(Candidate-Congruence)}$$

$h^{\parallel}$=`compositeHiddenOf`$(h)$, $m^{\parallel}$=`compositeMapOf`$(m)$

A common special case takes $K_C = K_A = K$ and $m = \mathrm{id}$; that sketch is already false, and the general form is what the positive theorem repairs under interface premises. Writing $\preceq$ without subscripts for the special case, the unconditional rule

$$\frac{C \preceq A}{C \parallel_K D \preceq A \parallel_K D} \qquad \text{(Invalid-Congruence)}$$

would be a lemma if hiding meant silence in every context. It is not.

This paper mechanizes a minimal counterexample and a matching repair. A concrete sender $C$ has a single action ping. Declared hidden, ping does not change $C$'s state or observation, so $C$ one-step refines a step-free abstract sender $A$. A receiver $D$ flips an observed Boolean on receive. Connect ping to receive. The composite $C \parallel_K D$ can take a synchronized step that changes the pair's observation from $(\mathsf{false}, \mathsf{false})$ to $(\mathsf{false}, \mathsf{true})$. The abstract composite $A \parallel_K D$ has no corresponding left step with which to synchronize. Treating the product step as hidden collapses the two composite states under the refinement map and forces $\mathsf{false} = \mathsf{true}$ by observation preservation. Treating it as visible leaves the abstract product stuck. Either way, (Invalid-Congruence) fails for this refinement and this composition operator. (The visible-classification failure is `visibleClassification_fails_on_hidden_ping_wire`.)

The failure is not an artifact of an eccentric hiding rule. The candidate relation is a *one-step refinement mapping with stuttering*: hidden steps preserve the abstract state; visible steps match exactly one abstract step; observations are preserved. It is *not* conventional weak simulation: there are no $\tau^*$-paths, no multi-step abstract matching, and no relational simulation witness beyond a state map. The composition operator is the obvious synchronized product: linked labels must move together, unlinked labels move independently. What the local rule does not see is that a hidden label may still be *connected*. Local abstract-state equality then coexists with a peer observation change.

That diagnosis suggests a repair rather than an abandonment of hiding. On the same labelled-transition fragment we prove a conditional congruence theorem: if no hidden label is connected, if the concrete and abstract wirings cover and reflect each other, and if the label map sends unconnected concrete labels to unconnected abstract labels, then left-component one-step refinement lifts through the product under the default classifiers of (Candidate-Congruence). The three premises are independently indispensable for that default lift. Dropping the first recovers the ping/receive counterexample (`pingReceive_violates_noHiddenBoundary`). Dropping the third, while keeping wiring coverage, admits a second counterexample in which a non-injective label map collapses an independent concrete action onto a connected abstract name (`CollidedAbstractWire.isolation_necessary_for_default_lift`). Dropping wiring coverage while keeping the other two admits a third (`ExtraAbstractWire.wiring_necessary_for_default_lift`).

A sibling result lives in a different model. Exact-action (strong) refinement of open systems with explicit ports, in which internal actions are ineligible for synchronization by construction, does lift through parallel composition under whole-wiring equivalence. We report that theorem because it is checked and because it shows that an interface-aware positive result is available once hiding through connections is forbidden structurally. It is not a proof of the weak one-step theorem, not residual RFC 0008 conditional congruence, and we do not treat it as one.

**Contributions.**

1. A Lean 4 mechanization of (Invalid-Congruence)'s failure: local sender refinement, an enabled synchronized step, and the absence of any observation-preserving one-step refinement of the composites (Theorems 9–13).

2. The diagnosis that local abstract-state equality is not contextual silence at a live connection, and the corresponding interface obligation I-NO-HIDDEN-BOUNDARY.

3. A checked one-step observation-safety lift on the same small model (Theorem 21), with positive controls and hypothesis-independence lemmas for isolation reflection and wiring coverage (Lemmas 25,26).

4. A restricted exact-action product congruence theorem in a stronger open-system model (Theorem 28), separated from the weak development and not residual C1.

The host development is NMLT, an Apache-2.0 research kernel for behavior-indexed types and evidence-carrying composition. The theorems of this paper concern the labelled-transition fragment in `NMLT.Core.Transition` and the modules cited in Appendix A. We do not claim a verified translation from NMLT source programs to these transition systems, nor congruence for capabilities, grades, or fairness, nor that Theorem 21 closes full RFC 0008 conditional congruence.

## 2 The model

Notation follows the Lean development in `NMLT.Core.Transition`.

**Definition 1** (LTS)**.** For types *Label* and *Obs*, a labelled transition system is a record

$$\langle \mathit{State},\ \mathit{init} : \mathit{State} \to \mathrm{Prop},\ \mathit{step} : \mathit{State} \to \mathit{Label} \to \mathit{State} \to \mathrm{Prop},\ \mathit{observe} : \mathit{State} \to \mathit{Obs} \rangle$$

(`NMLT.LTS`).

**Definition 2** (One-step refinement mapping with stuttering)**.** Fix LTSs $C$ and $A$ with a common observation type, a hiding map $h : \mathit{Label}_C \to \mathrm{Bool}$, and a label map $m : \mathit{Label}_C \to \mathit{Label}_A$. A witness of $C \preceq_{h,m} A$ (`WeakRefines`) is a state map $\mu$ such that:

- **init:** $C.\mathit{init}\, s$ implies $A.\mathit{init}\, (\mu\, s)$;
- **observe:** $C.\mathit{observe}\, s = A.\mathit{observe}\, (\mu\, s)$;
- **hidden step (stutter):** if $C$ steps $s \xrightarrow{\ell} t$ and $h(\ell) = \mathsf{true}$, then $\mu\, s = \mu\, t$;
- **visible step:** if $C$ steps $s \xrightarrow{\ell} t$ and $h(\ell) = \mathsf{false}$, then $A$ steps $\mu\, s \xrightarrow{m(\ell)} \mu\, t$.

A hidden concrete step is required to preserve the *abstract state*, not merely the concrete observation. There is no multi-step abstract matching: a visible concrete step corresponds to exactly one abstract step. In particular, this is *not* conventional weak simulation in the process-calculus sense (no $\tau^*$-closure, no relational witness relating states after silent sequences). It is closer to a one-step Abadi–Lamport refinement mapping with stuttering on hidden labels.

**Definition 3** (Synchronized parallel composition)**.** Product labels are $\mathsf{left}\,\ell$, $\mathsf{right}\,r$, or $\mathsf{sync}(\ell, r)$ (`ParallelLabel`). A connection $K$ is a relation on left and right labels (`Connection.linked`). The product $L \parallel_K R$ (`parallel`) has state space $L.\mathit{State} \times R.\mathit{State}$, componentwise initialization and observation, and steps:

- $\mathsf{left}\,\ell$ only if $\ell$ is not linked to any right label, the left component steps, and the right state is unchanged;

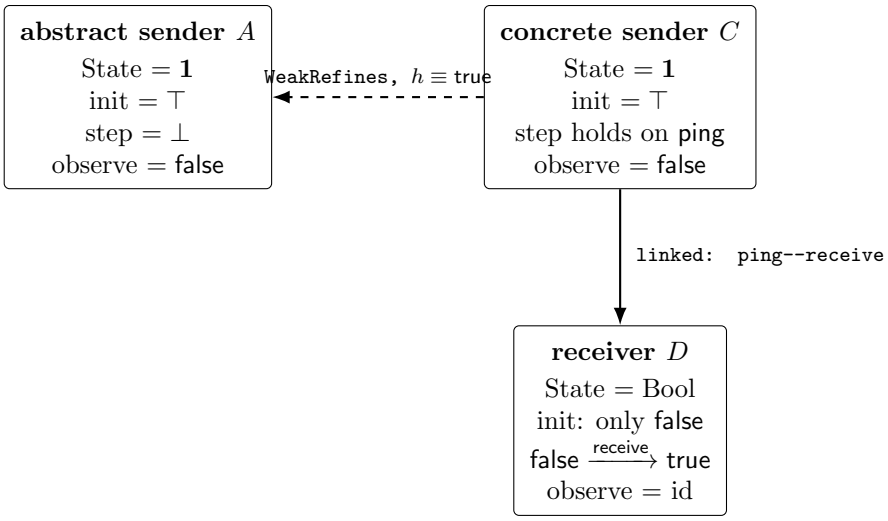



Figure 1: The three systems. Locally, hidden ping preserves $C$'s observation and maps to the unique abstract state. The connection still exposes ping to $D$.

- right $r$ symmetrically;
- sync$(\ell, r)$ only if $K$ links $\ell$ to $r$ and both components step.

Linked labels cannot move independently. Unlinked labels cannot synchronize.

This is a small model. It has no ports, directions, capabilities, or fairness. That is deliberate: the counterexample and the one-step lift should be checkable against the same four-field LTS, without an intervening encoding.

# 3 The counterexample

Let *SenderLabel* ::= ping and *ReceiverLabel* ::= receive. The constructions are those of `NMLT.Counterexamples.CompositionCongruence`.

**Definition 4** (Abstract sender $A$)**.** `abstractSender`: state **1**, every state initial, no steps, observation constantly false.

**Definition 5** (Concrete sender $C$)**.** `concreteSender`: state **1**, every state initial, a step for label ping to any post-state, observation constantly false.

**Definition 6** (Receiver $D$)**.** `receiver`: state Bool, only false initial; a step holds iff the label is receive, the pre-state is false, and the post-state is true; observation is the identity on Bool.

**Definition 7** (Connection and composites)**.** `connection` links ping to receive. Write $C \parallel_K D$ for `concreteComposite` and $A \parallel_K D$ for `abstractComposite`.

## 3.1 Local refinement

**Definition 8** (Sender hiding)**.** `senderHidden` hides every sender label. The label map is id.

**Theorem 9** (Standalone sender refinement)**.** *The structure* `senderRefinement` *inhabits*

*WeakRefines* $C$ $A$ *senderHidden* id.

*Proof.* Take $\mu(\_) = ()$. Initialization and observation are immediate. Every concrete step uses ping, which is hidden, and $\mu$ is constant, so the hidden-step obligation is reflexivity. The visible-step obligation is vacuous. □

Thus the premise of (Invalid-Congruence) holds for this $C$ and $A$.

### 3.2 The composite

**Definition 10** (Composite hiding)**.** `compositeHidden` hides synchronized product labels and leaves independent left and right labels visible:

$$\texttt{compositeHidden}(\mathsf{sync}(\_,\_)) = \mathsf{true},$$
$$\texttt{compositeHidden}(\mathsf{left}\_) = \texttt{compositeHidden}(\mathsf{right}\_) = \mathsf{false}.$$

**Theorem 11** (Concrete synchronization)**.**

$$(C \parallel_K D).\mathit{step}$$
$$((), \mathsf{false})\ \mathsf{sync}(\mathsf{ping}, \mathsf{receive})\ ((), \mathsf{true}).$$

*Proof.* $K$ links ping to receive; $C$ may step on ping; $D$ steps false → true on receive (`concrete-Synchronization`). □

**Lemma 12** (Endpoint observations)**.** $(C \parallel_K D).\mathit{observe}\,((), \mathsf{false}) = (\mathsf{false}, \mathsf{false})$ *and* $(C \parallel_K D).\mathit{observe}\,((), \mathsf{true}) = (\mathsf{false}, \mathsf{true})$*. In particular the second components differ.*

**Theorem 13** (No composite one-step refinement)**.** *There is no inhabitant of*

*WeakRefines* $(C \parallel_K D)$ $(A \parallel_K D)$ *compositeHidden* id.

*Equivalently,* ¬Nonempty(⋯) *as in* ***noCompositeRefinement****.*

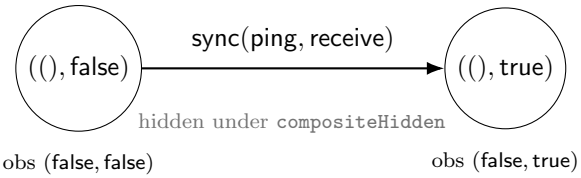
*Proof.* Suppose $R$ is such a refinement with state map $\mu$. Theorem 11 gives a $C \parallel_K D$ step on a hidden product label, so the hidden-step rule yields $\mu((), \mathsf{false}) = \mu((), \mathsf{true})$. Observation preservation for $R$, together with congruence of $(A \parallel_K D).\mathit{observe}$, implies

$$(C \parallel_K D).\mathit{observe}\,((), \mathsf{false}) = (C \parallel_K D).\mathit{observe}\,((), \mathsf{true}).$$

Projecting the second component contradicts Lemma 12. □

**Corollary 14** (Invalid-Congruence fails)**.** *The rule "$C \preceq A$ implies $C \parallel_K D \preceq A \parallel_K D$" is false for Definitions 2–3 and Example 4–7.*

*Remark* 15 (The visible reading also fails). If sync(ping, receive) is instead treated as visible, the abstract product still has no matching step: $A$ admits no left-component step with which to synchronize. The mechanized theorem `noCompositeRefinement` discharges the hidden reading used by "synchronization is silent." The visible reading is `visibleClassification_fails_on_hidden_ping_wire`.




Hidden-step rule $\Rightarrow \mu((), \mathsf{false}) = \mu((), \mathsf{true})$.
Observation preservation $\Rightarrow$ equal observations.
Second component: $\mathsf{false} = \mathsf{true}$. Contradiction.

Figure 2: The composite step used in Theorem 13.

# 4 Diagnosis

Definition 2's hidden-step clause constrains only the component state map $\mu$. Definition 3 allows the same concrete label to appear in a sync transition with a peer. Nothing in the local rule prevents a hidden action from changing a peer observation.

**Local state-map equality is not contextual silence.**

A tempting hygiene clause is that a hidden synchronization should map to a composite silent observation. That clause does not repair Example 4–7: even if the product label is classified hidden, $D$'s observation still changes from false to true, and Theorem 13 applies. The clause is withdrawn.

The failure is not that hiding is meaningless. It is that hiding was allowed to coexist with a live connection. The first repaired obligation is therefore:

**Definition 16** (No hidden boundary)**.** If $h(\ell) = \mathsf{true}$ under the refinement witness, then $\ell$ is not linked by the concrete connection $K_C$ to any peer label (`NoHiddenBoundary`).

**Lemma 17** (Independence of no-hidden-boundary)**.** *`NoHiddenBoundary` fails for `senderRefinement` and `connection` (`pingReceive_violates_noHiddenBoundary`).*

*Proof.* `senderHidden` hides ping, and `connection` links ping to receive, so ping is connected. □

# 5 Conditional one-step observation-safety congruence

Forming a product and lifting a refinement through a product are different judgments. The latter needs an explicit interface.

Write $H(c, d) = (\mu(c), d)$ for the left-lift of a state map $\mu$ across a fixed peer (`liftMap`). The default composite hiding map `compositeHiddenOf` classifies a left or synchronized product label as hidden exactly when the underlying left label is hidden, and treats peer-only steps as visible. The default composite label map `compositeMapOf` applies $m$ on the left component and the identity on the peer. These are exactly the classifiers in (Candidate-Congruence).

**Definition 18** (Wiring coverage)**.** A pair of connections $K_C, K_A$ is *covered* by maps $m$ and id (`WiringCovered`) when every concrete link maps to an abstract link, and every abstract link is the image of some concrete link.

**Definition 19** (Isolation reflection)**.** The maps *reflect isolation* (`IsolationReflects`) when every unconnected concrete left label remains unconnected after $m$: if $\ell$ has no $K_C$-partner, then $m(\ell)$ has no $K_A$-partner.

Coverage is a statement about the *wires*. Isolation is a statement about the *free names*. They coincide when $m$ is injective (`isolation_of_wiring_injective`); they come apart when $m$ collapses an independent concrete label onto a connected abstract label.

**Definition 20** (Interface compatibility)**.** A witness $R : C \preceq_{h,m} A$ is *interface-compatible* with peer $D$ and wirings $K_C, K_A$ (`InterfaceCompatible`) when Definition 16, Definition 18, and Definition 19 all hold.

Observation of a product is componentwise by Definition 3, so the usual observation-commutation premise is not an extra hypothesis in this fragment. Synchronization of labels commutes with $m$ because the default composite map applies $m$ on the left and id on the right.

**Theorem 21** (Small-model one-step observation-safety lift)**.** *In the model of Definitions 2–3, if $R$ is interface-compatible with $D$, $K_C$, and $K_A$ in the sense of Definition 20, then $H$ is a `WeakRefines` of the products under `compositeHiddenOf` and `compositeMapOf` (`weakConditionalCongruence_safety`).*

*Proof sketch.* The Lean proof is a case analysis on product labels.

1. **Init / observe.** Component initiality and $R$'s init and observe rules lift pairwise.
2. **Hidden left.** Definition 16 makes the label unconnected, so the product permits a left-only step. $R$'s hidden-step rule gives $H(c, d) = H(c', d)$ with the peer fixed. A hidden sync is impossible: a connected label cannot be hidden.
3. **Visible independent left.** $R$'s visible-step rule supplies an abstract left step. Definition 19 keeps the image unconnected on $K_A$, so the abstract product also permits the left-only step.
4. **Independent peer.** Reuse the same $D$ step. Wiring reflection sends an abstract peer-link to a concrete one, so independence is preserved, and the left coordinate of $H$ is unchanged.
5. **Synchronization.** Definition 16 forces the concrete left label visible. $R$'s visible-step rule and wiring coverage supply a matching abstract sync with the same peer transition.

□

The theorem is an observation-safety result about one-step refinement mappings on this LTS fragment. It does not transport liveness, divergence, capabilities, or grades. It does not close full RFC 0008 conditional congruence.

**Corollary 22** (Finite observation-trace inclusion)**.** *If $C \preceq_{h,m} A$ (`WeakRefines`), every finite observation trace of $C$ is a stutter-expansion of a finite observation trace of $A$ from the mapped start state (`weakRefines_finite_observation_trace_inclusion`); in particular this holds for product observations under Theorem 21. Finite traces only: not LTL, infinite words, fairness, or liveness.*

*Proof sketch.* By induction on the concrete run: the observe clause matches the initial observation; a hidden step repeats the current abstract observation; a visible step takes exactly one mapped abstract step. □

### 5.1 Positive and negative controls

**Lemma 23** (Visible ping/receive)**.** *The same connection as Example 4–7, with* ping *classified visible and with an abstract sender that can take* ping*, admits a product one-step refinement (`VisibleSync.visibleSync_productRefinement`).*

**Lemma 24** (Empty wiring and hidden $\tau$)**.** *A hidden left $\tau$ over the empty connection admits a product one-step refinement (`EmptyWiringTau.emptyWiring_productRefinement`).*

Lemma 23 is the ping/receive system with the offending hiding bit flipped: the wiring is unchanged, the local refinement is now a strong one, and the lift goes through. Lemma 24 is the complementary case in which hiding is genuine because there is nothing to connect.

**Lemma 25** (Isolation reflection is indispensable for the default lift)**.** *There exist systems and wirings such that Definition 16 and Definition 18 hold, Definition 19 fails, and no `WeakRefines` of the products exists under the default composite classifiers (`CollidedAbstractWire.isolation_necessary_for_default_lift`).*

**Lemma 26** (Wiring coverage is indispensable for the default lift)**.** *There exist systems and wirings such that Definition 16 and Definition 19 hold, Definition 18 fails (an abstract wire on a label outside the image of $m$), and no `WeakRefines` of the products exists under the default classifiers (`ExtraAbstractWire.wiring_necessary_for_default_lift`; axioms: none).*

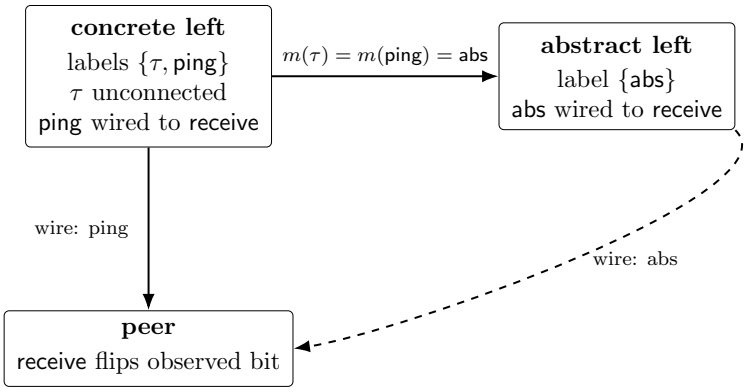
*Proof sketch.* The concrete left component has labels $\tau$ and ping; the abstract left component has a single label abs; both concrete labels map to abs. Only ping is wired to a peer receive, so $\tau$ is independent. Nothing is hidden, so Definition 16 is vacuous. Wiring coverage holds because the unique abstract wire reflects to ping. Isolation fails at $\tau$: $m(\tau) =$ abs is connected. The concrete product can take a left-only $\tau$ step; the abstract product cannot take a left-only abs step. The default visible-left obligation is therefore unsatisfiable. □

Together, Theorem 13 and Lemmas 17, 25, and 26 say that each named premise of Theorem 21 is sharp for the default lift in the hypothesis-independence sense: omitting that premise alone, while keeping the others (or the ping/receive pattern for NHB), admits a counterexample under `compositeHiddenOf`/`compositeMapOf`. We do not claim necessity in any stronger, model-universal sense.

*Conjecture* 27 (Residual weak congruence)*.* Theorem 21 does not yet give congruence for capabilities, grades, fairness, or the port/assumption model of Theorem 28 below. Deriving Definition 19 from Definition 18 without injectivity of $m$ remains unsound, as Lemma 25 shows. Closing residual C1 / full RFC 0008 CONDITIONAL-CONGRUENCE remains open.

## 6 An exact-action sibling

The repository also contains a *different* refinement model: exact-action (strong) refinement of open systems with explicit ports, assumptions and guarantees, and whole-wiring equivalence (`NMLT.Behavior.OpenComposition`). Internal actions in that model are ineligible for synchronization by construction, so the ping/receive pattern cannot be expressed as a hidden connected action. The results of this section are therefore not residual obligations of Conjecture 27; they live in another formalism.




Coverage holds (abstract wire reflects to ping). Isolation fails at $\tau$: concrete can take left-only $\tau$; abstract cannot take left-only abs.

Figure 3: Isolation counterexample (Lemma 25). Coverage fails for a non-injective map.

**Theorem 28** (Exact-action product congruence)**.** *In the OpenComposition model, with whole-wiring equivalence and the stated composability premises,* `StrongRefinement.compositionCongruence` *holds: left-component refinement lifts through parallel composition, and product receptiveness and composability are preserved as stated in the development. (Self-contained reading: exact matching of actions, no weak hiding classifier, ports rather than free labels, and internal actions barred from synchronization.)*

Theorem 28 is not a proof of Theorem 21 or of Conjecture 27, and it is not residual C1. By the module's own statement it claims nothing about weak hiding, label maps beyond the strong model, capability partition, grades, or liveness. We include it as evidence that an interface-aware positive lifting theorem is available once hiding through connections is ruled out structurally.

# 7 Related work

**Refinement mappings and stuttering.** Abadi and Lamport's refinement mappings [1] and Lamport's TLA [5] treat implementation steps as abstract stuttering under a state map. Definition 2's hidden-step rule is in that spirit: hidden steps preserve abstract *state*. The counterexample shows that port synchronization is an additional obligation those local rules do not discharge. A stuttering step that is still connected is not a stutter of the composite.

**Weak equivalences and congruence formats.** Classical weak bisimulation and weak simulation [8, 11] close silent transitions under $\tau^*$ and ask for matching after silent sequences. Congruence formats for weak equivalences (e.g. the tyft/tyxt and ntyft/ntyxt tradition, and later rule formats for weak bisimulation) [3, 10] study syntactic conditions under which a weak equivalence is a congruence for a process language's operators. Our contribution is complementary and narrower: we fix an *already chosen* one-step refinement mapping (no $\tau^*$), fix a synchronized product, and mechanize the failure of unconditional contextual lift when hiding coexists with a live connection—together with hypothesis-independence controls for a conditional repair. We do not propose a new congruence format.

**I/O automata and simulations.** I/O automata distinguish input, output, and internal actions and prove composition theorems under compatibility and receptiveness [6, 7]. Forward simulations in that tradition are not expected to lift through composition without those premises. Our result is consistent with that lesson: the failure appears when compatibility-style premises are omitted from a TLA-style hiding rule. We do not propose a new automaton model.

**Interface automata.** de Alfaro and Henzinger's interface automata [2] make composition a partial operation, failing when an output is not accepted. The moral is again that composition is not automatic. Theorem 21 takes the complementary route of stating the missing premises rather than making the product partial.

**Process calculus hiding.** CCS restriction and hiding [8], and CSP concealment [4], remove names from the interface. The mistake in (Invalid-Congruence) is to hide a name that is still in the interface of the product. Once ping is connected it is not an internal name of $C \parallel_K D$, regardless of $C$'s local classification.

**Stutter-invariant temporal logic.** Peled and Wilke [9] characterize stutter-invariant LTL by the absence of next. That motivates separating action-level claims from observation transport. It does not, by itself, police connected hidden labels.

# 8 Artifact

The definitions and proofs are checked in Lean 4.30.0. The trusted computing base for Theorems 9–21 is the Lean kernel, the four-field LTS of `NMLT.Core.Transition`, and the modules in Appendix A. There is no `sorry` and no project-defined axiom. Lean's propositional extensionality `propext` appears in Theorem 13 and Theorem 21; the independence lemmas of Section 4 and Lemmas 25,26 are axiom-free. Mathlib and Aeneas are dependencies of the broader NMLT workspace and are not on the TCB of these theorems.

**Local tree vs published `main`.** As of this draft, Theorem 21 (`weakConditionalCongruence_safety`) and its independence controls live in the local freeze commit `451001b`, which is ahead of the published GitHub `main` tip `0417f6e`. That SHA is a local-only artifact identifier until the authors push and tag; it is not yet a remote ref.

```
artifact-SHA = 451001b
451001b5697cfc7ec3e422412f0268a84a72da95
(local; not pushed)
```

Do not treat `0417f6e` alone as containing Theorem 21.

**Reproduce.** From a checkout that contains the cited modules, on Lean 4.30.0:

```
cd mechanization/lean
lake build NMLT
# axiom audit (representative):
lake env lean --run <<'EOF'
  -- or open the modules and evaluate:
  -- #print axioms weakConditionalCongruence_safety
```

```
  -- #print axioms noCompositeRefinement
  -- #print axioms StrongRefinement.compositionCongruence
EOF
```

Equivalently, after `lake build NMLT`, inspect the `#print axioms` lines already present in

- `Behavior/WeakConditionalCongruence.lean`
- `Counterexamples/CompositionCongruence.lean`
- `Behavior/OpenComposition.lean`
- or the summary `papers/hidden-silence/axiom-audit-2026-08-27.md`

The host repository is NMLT, distributed under Apache License 2.0.[1] Appendix B records a `#print axioms` audit from a clean `lake build`. A surface-language sketch of the three systems, not part of the frozen example corpus and not a checked composition claim, is `examples/paper1/hidden_ping_receive.nmlt`.

## 9 Conclusion

A hidden action is not contextually silent if it is still connected. We mechanized the resulting failure of unconditional one-step refinement congruence under synchronization, and we mechanized a matching conditional observation-safety lift on the same small model: no hidden connected labels, covered wirings, and a label map that preserves independence. The exact-action theorem shows that a stronger model can rule the failure out by construction; it is a different model, not residual C1. What remains is to carry the one-step theorem up through resources, grades, and fairness, and to connect these transition systems to the surface language, without weakening the interface conditions that the counterexamples show are independently indispensable for the default lift.

## A Lean pointers

**LTS / map / product** `NMLT.LTS`, `WeakRefines`, `parallel` in `Core/Transition.lean`.

**Finite obs. traces** `weakRefines_finite_observation_trace_inclusion` in `Core/FiniteObservationTrace.lean`.

**Counterexample** `senderRefinement`, `concreteSynchronization`, `noCompositeRefinement`, `visibleClassification_fails_on_hidden_ping_wire` in `Counterexamples/CompositionCongruence.lean`.

**One-step lift** `InterfaceCompatible`, `weakConditionalCongruence_safety` in `Behavior/WeakConditionalCongruence.lean`.

**Indep. (NHB)** `pingReceive_violates_noHiddenBoundary`, `visiblePing_breaks_senderRefinement` (same module).

**Indep. (isolation)** `CollidedAbstractWire.isolation_necessary_for_default_lift`.

**Indep. (wiring)** `ExtraAbstractWire.wiring_necessary_for_default_lift`.

**Positive controls** `VisibleSync.visibleSync_productRefinement`, `EmptyWiringTau.emptyWiring_productRefinement`.

**Exact-action lift** `StrongRefinement.compositionCongruence` in `Behavior/OpenComposition.lean`.

[1] `https://github.com/agentcarlosian/NMLT`

## B Axiom audit

Recorded from `lake build NMLT` of the mechanization on Lean 4.30.0 (27 August 2026). Policy: no `sorry`, no project-defined axioms. Lean's foundational `propext` is an accepted TCB entry when it appears. Artifact SHA: exttt451001b (local freeze; not pushed).

| Claim | Lean declaration | Axioms |
|---|---|---|
| Thm. 9 | `senderRefinement` | none |
| Thm. 11 | `concreteSynchronization` | none |
| Thm. 13 (lemma) | `compositeRefinementImpossible` | `[propext]` |
| Thm. 13 | `noCompositeRefinement` | `[propext]` |
| Rem. 15 | `visibleClassification_fails_on_hidden_ping_wire` | none |
| Lem. 17 | `pingReceive_violates_noHiddenBoundary` | none |
| Thm. 21 | `weakConditionalCongruence_safety` | `[propext]` |
| Cor. 22 | `weakRefines_finite_observation_trace_inclusion` | none |
| Lem. 23 | `visibleSync_productRefinement` | `[propext]` |
| Lem. 24 | `emptyWiring_productRefinement` | `[propext]` |
| Lem. 25 | `isolation_necessary_for_default_lift` | none |
| Lem. 26 | `wiring_necessary_for_default_lift` | none |
| Thm. 28 | `compositionCongruence` | none |

The impossibility proof uses propositional extensionality to compare observations after identifying equal abstract states. The isolation and wiring independence lemmas, the local sender refinement, and the visible-classification failure are axiom-free.

## References


[1] M. Abadi and L. Lamport. The existence of refinement mappings. *Theoretical Computer Science*, 82(2):253–284, 1991.

[2] L. de Alfaro and T. A. Henzinger. Interface automata. In *Proceedings of ESEC/FSE 2001*, pp. 109–120. ACM, 2001.

[3] J. F. Groote and F. Vaandrager. Structured operational semantics and bisimulation as a congruence. *Information and Computation*, 100(2):202–260, 1992.

[4] C. A. R. Hoare. *Communicating Sequential Processes*. Prentice-Hall, 1985.

[5] L. Lamport. *Specifying Systems*. Addison-Wesley, 2002.

[6] N. A. Lynch and M. R. Tuttle. Hierarchical correctness proofs for distributed algorithms. In *Proceedings of the Sixth Annual ACM Symposium on Principles of Distributed Computing (PODC)*, pp. 137–146. ACM, 1987.

[7] N. A. Lynch and F. W. Vaandrager. Forward and backward simulations. Part I: Untimed systems. *Information and Computation*, 121(2):214–233, 1995.

[8] R. Milner. *Communication and Concurrency*. Prentice-Hall, 1989.

[9] D. Peled and T. Wilke. Stutter-invariant temporal properties are expressible without the next-time operator. *Information Processing Letters*, 63(5):243–246, 1997.

[10] I. Ulidowski and I. Phillips. Ordered SOS rules and weak bisimulation. In *CONCUR 2002*, LNCS 2421, pp. 172–186. Springer, 2002.

[11] R. J. van Glabbeek. The linear time – branching time spectrum II: the semantics of sequential systems with silent moves. In *CONCUR '93*, LNCS 715, pp. 66–81. Springer, 1993.